Drone Pathfinder: AI Drone Navigation Using Reinforcement Learning

Daniel Fridström[1], Hans Fredö[2]

Abstract
This project investigates a reinforcement learning AI model for a drone agent's navigation in a 3D simulated environment. The objective is to train the drone agent to navigate an obstacle course and reach a sequence of five checkpoints while avoiding collisions. The AI model uses Proximal Policy Optimization (PPO) and the project uses a curriculum learning approach. Meaning that each level from 1 to 10 increases its difficulty for each level. The Godot Engine is used for this project to simulate the 3D environment and to communicate the reward scores to the AI. Python is used for training the AI. Performance is tracked and evaluated using thresholds and metrics such as task completion, checkpoint progression, collision behaviour, completion time and navigation efficiency.

Source code: *[GitHub project repository]*
Video 1: *[https://youtu.be/knbbf1tdBZo]* **Video 2**: *[https://youtu.be/jq0QMd45nq8]*

Authors
[1] *Media Technology Student at Linköping University, danfr755@student.liu.se*
[2] *Media Technology Student at Linköping University, hanfr829@student.liu.se*

Keywords: Reinforcement Learning — PPO — Godot — Python — Curriculum Learning — Drone Navigation

Contents

1 Introduction 1
2 Theory 2
2.1 Reinforcement Learning . . . 2
2.2 Proximal Policy Optimization . . . 2
2.3 Godot . . . 2
2.4 Curriculum Learning . . . 2
3 Method 2
3.1 System Overview . . . 2
3.2 Environment and Task . . . 2
3.3 Observations . . . 2
3.4 Actions . . . 2
3.5 Reward Function . . . 3
3.6 Training Procedure . . . 4
3.7 Thresholds . . . 4
3.8 Evaluation Metrics . . . 4
4 Results 4
4.1 Untrained Agent Performance . . . 4
4.2 Comparison of Untrained and Trained Agents . . . 5
4.3 Summary . . . 5
5 Discussion 5
5.1 Effect of Curriculum Learning . . . 5
5.2 Limitations and Future Work . . . 5
6 Conclusion 6
References 6

1. Introduction

Traditional navigation systems may rely on manually designed movement rules or explicit path planning algorithms. These approaches can require substantial effort when the environment becomes more complex. However, reinforcement learning (RL) provides an alternative by improving the agent's behavior through interaction with an environment and feedback from a reward function [1].

The purpose of this project is to develop a 3D simulation with a drone agent that learns to navigate an obstacle course using RL. The project uses Proximal Policy Optimization (PPO), which is used to optimize the policy for the drone agent. The project is implemented by using the Godot [2] engine and Python [3] in Visual Studio.

The project addresses three research questions:

1. Can the simulated drone agent learn an effective navigation policy by using PPO?
2. How does performance for the drone agent change as the course difficulty increases?
3. How effectively can the trained drone agent reach the checkpoints while avoiding collisions?

2. Theory

2.1 Reinforcement Learning

Reinforcement learning (RL) is used in the project and is a machine learning approach where an agent learns by trial and error [1]. The agent interacts with the simulated environment and then receives either a reward or a penalty based on its actions. This teaches the agent which actions are more effective.

2.2 Proximal Policy Optimization

The specific version of RL the project is using is Proximal Policy Optimization (PPO) [4], which is the training system implemented in this project using the Stable-Baselines3 library in Python [5]. PPO is a reinforcement learning algorithm that learns which actions an agent should take in different situations. It updates the agent's policy based on its experience while also limiting how much the policy can change with each update. This helps stabilize training.

PPO is used in this project to improve the drone's navigation behavior through a trial-and-error training approach. The goal is for the drone to reach checkpoints more effectively while avoiding obstacles.

2.3 Godot

Godot is used in this project to be able to simulate a 3D environment with a drone. It also communicates with the Python code that is responsible for the training system. Godot sends the rewards and penalty scores to the Python code, which in turn updates the agent's policy. The Python library Gymnasium [6] is used to provide the interface between the training system and the 3D simulation environment in Godot.

2.4 Curriculum Learning

A curriculum learning approach is used in the project, and it is a training approach in which tasks become progressively more difficult. The drone agent begins with a simpler environment and gradually encounters harder situations. The performance across the levels can then be used to evaluate how well the agent handles increasing difficulty.

3. Method

3.1 System Overview

The project consists of a 3D environment simulated in Godot that also sends the reward or penalty score to the separate training system in Python. The training system takes those score points and calculates the rewards and learning targets, updates the policy, and evaluates the resulting behavior. The learned policy is then used to select actions during subsequent environment interactions.

The drone agent repeatedly trains on the same level until it reaches the set threshold for it to be allowed to train on the next level. The drone agent training is done when it reaches the final tenth level and has, after 100 training tries, at least a success rate of 70 percent and a path efficiency of 80 percent.

Figure 1 shows the actual simulated 3D environment.

3.2 Environment and Task

Each iteration represents one attempt by the drone agent to navigate the course in the current level. The drone agent is trained to navigate a 3D obstacle course, and its goal is to reach five checkpoints. The obstacles restrict the available path decisions and require the agent to balance the forward progression (towards the checkpoint) with collision avoidance. The current run the agent is on is only terminated if it successfully completes the current course or if it collides with an obstacle.

The 3D environment provides the agent with experience under different levels of difficulty. Thus, by using the curriculum learning approach, it can train the agent and teach it to assess navigation with the same basic task but in progressively more challenging situations.

3.3 Observations

At each decision step, the drone agent receives an observation that describes its current situation while navigating the course. The PPO policy uses this information to select what it thinks is the best course of action to take. The drone agent receives 72 observation values describing its state, target, and surroundings. These values are grouped as follows:

- Drone position and velocity (6)
- Relative target position and horizontal distance (4)
- Checkpoint progress and curriculum level (2)
- Target direction (3)
- Obstacle sensors (18)
- Remaining route distance (1)
- Arena boundary distances (6)
- Nearest obstacle distance (1)
- Absolute target position and direct distance (4)
- Route progress (1)
- Previous action and movement alignment (2)
- Candidate-action evaluations (16)
- Action memory (8)

3.4 Actions

The drone agent has discrete actions and can choose between eight movement actions. These actions determine how the drone moves through the environment. The observations tell the agent about its current situation, while the actions define how the drone agent can move towards the checkpoints. These are the eight discrete movement actions the drone agent can take:

1. Forward (towards the target).
2. Forward-left.
3. Forward-right.
4. Forward-up.
5. Forward-down.
6. Move left.
7. Move right.
8. Move backwards.

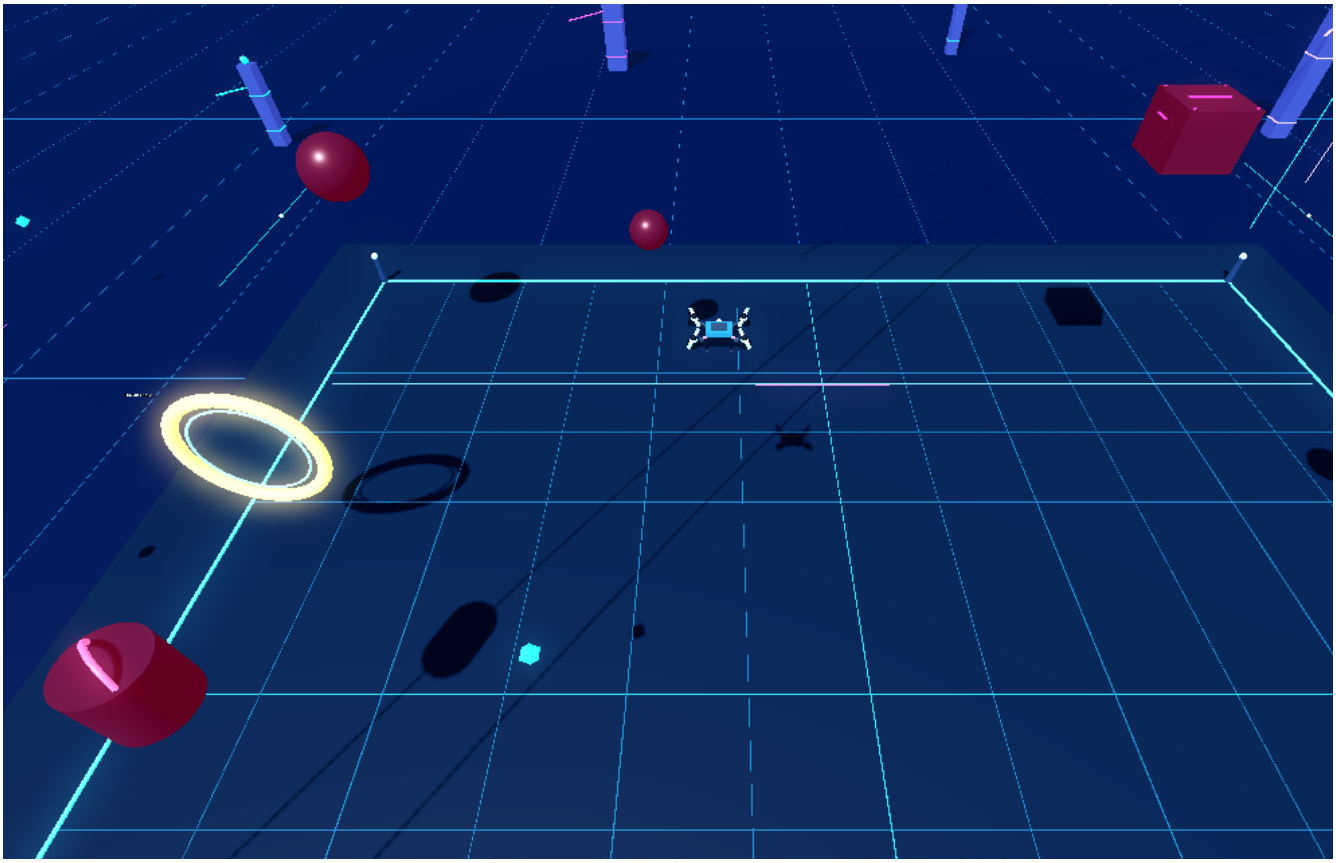


Figure 1. The Drone Pathfinder 3D simulation environment (level 1 with 5 obstacles).

3.5 Reward Function

The reward function is implemented in Godot and sent to the Python training system after each action. PPO uses these reward scores to learn which actions it takes are bad and good. This means the agent learns which actions improve the navigation performance.

These are the main reward components:

- **Progress** (r_{progress})**:** Between -2.5 and $+2.5$, based on the distance to the current target.
- **Route progress** (r_{route})**:** Between -2.0 and $+2.0$, based on changes in the remaining route distance.
- **Obstacle penalty** (p_{obstacle})**:** A distance-dependent penalty applied within 6.0 units of an obstacle, with a stronger penalty within 2.5 units.
- **Time penalty** (p_{time})**:** A penalty of -0.04 for each normal decision step.
- **Inactivity penalty** ($p_{\text{inactivity}}$)**:** If the drone moves less than 0.05 units during a step, then an additional penalty -0.5. Otherwise, no inactivity penalty is applied.
- **Checkpoint reward** ($r_{\text{checkpoint}}$)**:** A reward of $+150$ for each checkpoint reached, added separately from the normal step reward.
- **Terminal rewards** (r_{terminal})**:** A penalty of -300 for a collision, leaving the arena, or getting stuck. Completing the course gives a reward of $800+350E$.

You can write the reward calculation simplified for a normal decision step as:

$$r_{\text{step}} = r_{\text{progress}} + r_{\text{route}} - p_{\text{time}} - p_{\text{obstacle}} - p_{\text{inactivity}}. \tag{1}$$

The inactivity penalty is defined as:

$$p_{\text{inactivity}} = \begin{cases} 0.5, & \text{if } d_{\text{step}} < 0.05, \\ 0, & \text{if } d_{\text{step}} \geq 0.05. \end{cases} \tag{2}$$

Here, d_{step} is the distance the drone moves during one decision step in the simulation.

When the iteration is terminated, an additional reward is added and handled separately:

$$r_{\text{terminal}} = \begin{cases} -300, & \text{collision or getting stuck,} \\ 800+350E, & \text{course completed.} \end{cases} \tag{3}$$

Here, E represents the route efficiency, ranging from 0 to 1. The final reward calculation can then be simplified to:

$$r_{\text{final}} = r_{\text{step}} + r_{\text{terminal}} + r_{\text{checkpoint}}. \tag{4}$$

The final reward is the sum of the step reward, any checkpoint rewards, and any terminal reward.

3.6 Training Procedure

The drone is trained using PPO in Python, while Godot runs the simulation and provides observations, rewards, and iteration termination. The training process repeats the following steps:

1. Initialize the Godot environment and PPO policy.
2. Reset the environment and obtain the initial observation.
3. Select a movement action using the current policy.
4. Execute the action in Godot and receive the next observation, reward, and termination status.
5. Collect experience and calculate returns and advantages.
6. Update the policy and value function using PPO.
7. Track performance and curriculum progression.
8. Repeat until the training timestep budget is reached.

3.7 Thresholds

The curriculum contains ten levels, with obstacles increasing from 5 in the easiest level to 60 in the hardest level. There's a set threshold that the agent must meet before it is allowed to proceed to the next level, and the threshold consists of three conditions.

The first condition it needs to achieve is a success rate of at least 70%, measured over 100 iterations. The second condition is to reach an average of at least 4 out of 5 checkpoints. The third condition is to achieve an average path efficiency of at least 65% over 100 iterations. These thresholds help to make sure that the drone performs consistently before advancing onto a more difficult level.

3.8 Evaluation Metrics

The drone agent's performance is evaluated using these metrics:

Success rate The percentage of iterations in which the drone agent completes the course.

Mean checkpoints The average number of checkpoints reached out of five.

Path efficiency Measures how efficiently the drone agent flew through the course (higher values indicate better efficiency).

Iteration length The number of decision steps and the simulation time taken before an iteration ends.

Reward The total reward collected during an iteration, which indicates how well the agent performs according to the reward function.

Figure 2 shows an example of the training output at Level 10 for a drone agent. Performance is measured over a rolling window of 100 completed iterations to determine whether the drone agent meets the requirements for advancing to the next level. In this example, the success rate is 66.0%, below the required 70%. The mean number of checkpoints is 4.18 out of 5, while the mean path efficiency is 92.2%, above the required 65%. The agent meets the checkpoint and path efficiency thresholds but not the success rate threshold in this example and thus is not allowed to advance to the next level. Since the drone agent is on the last level, it isn't allowed to finish its training until it succeeds instead.

The output also shows a mean reward of 1463.76 and a mean iteration length of 191.8 decision steps, corresponding to 13.1 seconds of simulated time. The best iteration in this PPO batch achieved a reward of 2309.69 and completed all five checkpoints. However, this single iteration does not represent the agent's overall performance, which is better reflected by the rolling-window metrics.

```
======================================================================
AI TRAINING | PPO ITERATION 66 | 568,459 TOTAL STEPS
======================================================================
LEVEL: 10/10 | obstacles: 60
LEVEL WINDOW: 100/100 completed episodes
  success              : 66.0% / 70% required
  mean checkpoints     : 4.18 / 5 / 4.00 required
  mean path efficiency : 92.2% / 65% required

MEAN — episodes completed in this PPO batch
  reward/score         : +1463.76
  lifetime             : 191.8 RL steps (13.1s)
  checkpoints          : 3.95 / 5
  path efficiency      : 93.8%
  success rate         : 61.9%

BEST — inside this PPO batch
  best reward          : +2309.69 (episode 1280, PPO iter 66)
  longest lifetime     : 341 steps (22.8s) (episode 1282, PPO iter 66)
  most checkpoints     : 5 / 5 (episode 1267, PPO iter 66)
  fastest finish       : 138 steps (10.1s) (episode 1267, PPO iter 66)
  best path efficiency : 100.0% (episode 1266, PPO iter 66)

TOP 5 EPISODES — this PPO batch, ranked by reward
  #1 ep  1280 | iter  66 | reward +2309.69 | life  227 | CP 5/5 | FINISH
  #2 ep  1270 | iter  66 | reward +2308.36 | life  282 | CP 5/5 | FINISH
  #3 ep  1282 | iter  66 | reward +2288.98 | life  341 | CP 5/5 | FINISH
  #4 ep  1284 | iter  66 | reward +2287.94 | life  268 | CP 5/5 | FINISH
  #5 ep  1279 | iter  66 | reward +2260.60 | life  257 | CP 5/5 | FINISH

GOAL LEARNING DETECTED — checkpoints/finishes are appearing.
```

Figure 2. Training output for a drone agent at Level 10, which contains 60 obstacles. The output shows the rolling-window metrics and performance in one PPO batch.

4. Results

The results show that training with PPO improved the drone agent's navigation performance.

The following sections present the untrained agent's performance across all ten levels and compare the trained and untrained agents at three selected levels. The trained agent achieved higher path efficiency, shorter episode durations, and success rates at all three levels.

4.1 Untrained Agent Performance

Table 1 on the next page shows the performance of the untrained agent as it progresses through the curriculum of the ten levels.

The untrained agent was able to reach all five checkpoints in some iterations. This shows that the agent's performance varied across all the levels. The results show that the agent could complete the course in some attempts without prior

Table 1. Untrained agent performance across curriculum levels.

Level	Obstacles	Success (%)	Mean time (s)
1	5	64.0	20.5
2	11	68.0	18.4
3	17	68.0	17.4
4	23	67.0	17.5
5	29	70.0	18.7
6	36	70.8	15.5
7	42	67.4	17.1
8	48	70.0	15.7
9	54	75.6	13.8
10	60	76.5	16.6

training, but these measurements do not establish that it had learned a reliable navigation policy yet.

4.2 Comparison of Untrained and Trained Agents

To investigate if the training actually works for the agent over the curriculum, the untrained agents' success, efficiency, and duration during the iteration was compared with those of a trained agent at levels 1, 5, and 10.

Table 2. Untrained and trained agent performance at levels 1, 5, and 10.

Level	Metric	Untrained	Trained
1	Success (%)	64.0	96.9
	Mean path efficiency (%)	75.0	95.6
	Duration (s)	20.5	15.7
5	Success (%)	70.0	76.4
	Mean path efficiency (%)	82.0	93.5
	Duration (s)	18.7	12.7
10	Success (%)	76.5	87.2
	Mean path efficiency (%)	88.0	93.8
	Duration (s)	16.6	13.1

At level 1, the trained agent achieved a higher success rate, mean path efficiency, and iteration duration. The trend continues in level 5 and level 10.

4.3 Summary

The comparison shows that the trained agent completed iterations in less time than the untrained agent at all three selected difficulty levels. Overall, the results provide evidence that the training improved the drone agent's path efficiency and reduced mean episode duration, meaning it affected the drone agent's navigation behavior in a positive way.

5. Discussion

The project combines PPO with curriculum learning to address the challenge of learning navigation in a 3D obstacle course. This allows the drone agent to improve through a trial-and-error approach instead of manually implemented navigation rules. Through repeated interactions with the 3D simulation, the drone agent learns which actions lead to a better outcome. The results show that the trained agent performed better than the untrained agent at levels 1, 5, and 10. It achieved higher success rates and path efficiency while also completing episodes in less time. This suggests that combining PPO with curriculum learning helps an agent to develop a more efficient navigation policy.

5.1 Effect of Curriculum Learning

Increasing obstacle density gradually is intended to make the initial learning problem more manageable. The results show that the trained agent performed better than the untrained agent at all three evaluated levels.

These results are consistent with the idea that training helps the agent navigate more effectively as obstacle density increases.

5.2 Limitations and Future Work

One limitation of the project is that the drone agent's performance depends on the reward function. The rewards and penalties influence which behaviours the drone agent learns. So maybe there is a more suitable balance between reaching checkpoints, avoiding obstacles, and moving efficiently. Meaning, if you were to introduce another threshold, reward, or tweak the current threshold and reward values, maybe it would improve it more than what the current version does in the project.

Another limitation is that the project focuses on navigation within a simple 3D simulated environment. The results therefore do not establish how the trained drone agent would perform in a real-world environment, where physical dynamics such as wind and gravity would affect the training in the simulation.

So if we had more time to continue this project, we would work on those two limitations to see how they would affect the training system with PPO on the drone agent in the new, more realistic 3D simulated environment.

6. Conclusion

This project successfully implements a reinforcement-learning system in Python for training a simulated drone to navigate a 3D obstacle course in Godot. The training system uses Proximal Policy Optimization (PPO) together with curriculum learning. The results shows that the training approach improved the drone agent's navigation performance. However, further experiments could investigate how different reward settings, physics settings, and training configurations affect performance, as well as how well the learned policy generalises to different obstacle arrangements.

References

[1] P. Dell'Acqua. Reinforcement learning – lec. 10. Lecture, ITN, Linköping University. Accessed: 18 October 2026. Available at: https://dellacqua.se/education/courses/tnm095/lectures.html.

[2] Godot Engine. *Godot Engine*. Godot Foundation. Accessed: 18 October 2026. Available at: https://godotengine.org/.

[3] Python Software Foundation. *Python Documentation*. Accessed: 18 October 2026. Available at: https://docs.python.org/3/.

[4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint:1707.06347*, 2017. Accessed: 18 October 2026. Available at: https://arxiv.org/abs/1707.06347.

[5] Stable-Baselines3 contributors. *Stable-Baselines3 Documentation*. Accessed: 18 October 2026. Available at: https://stable-baselines3.readthedocs.io/en/master/.

[6] Farama Foundation. *Gymnasium Documentation*. Accessed: 18 October 2026. Available at: https://gymnasium.farama.org/.